

HPC-ARC: Intelligent AI Benchmark Suite for HPC Performance Engineering

Interactive Reasoning Framework for Genuine AI Intelligence
Measurement

Anja Gerbes and Julian Kunkel

GWDDG - Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen
University of Göttingen

October 7, 2026

anja.gerbes@gwdg.de and julian.kunkel@gwdg.de

<https://github.com/gwdg/hpc-agentic-ai-workloads>

What is HPC-ARC About?

One-Sentence Summary

HPC-ARC is a benchmark suite that measures whether AI systems can perform **HPC performance engineering** — and how efficiently they learn it.

The Name

- ▶ **ARC** = **A**bstraction and **R**easoning **C**orpus: AI reasoning benchmark (arcprize.org); tests genuine intelligence, not memorization
- ▶ **HPC-ARC** = **H**igh-**P**erformance **C**omputing **A**bstraction and **R**easoning **C**hallenge: transfers this idea to HPC code optimization

Our Goal

- ▶ **Primary**: Find out whether AI genuinely *understands* HPC performance engineering
- ▶ Code optimization is the **vehicle** for measurement, not the end in itself
- ▶ Side effect: better optimized code

Four-phase interactive reasoning: Explore → Hypothesize → Execute → Generalize

Key Terms

- ▶ **Benchmark** = task given to the AI agent (e.g., optimize a kernel); evaluation = **executing** the (AI-)optimized application (GROMACS, LAMMPS) and **measuring** its performance
- ▶ **Environment** = target **hardware** (CPU, caches, NUMA) plus **software stack** (compilers, libraries, tools) the agent must discover
- ▶ **Interactive** = in ARC-AGI (ARC for **A**rtificial **G**eneral **I**ntelligence), agents explore game-like environments; here, the agent likewise **acts** (compiles, profiles, runs) and **learns from feedback** (measurements) — like an HPC expert

Three Critical Challenges:

- ▶ **No Standardization:** No unified benchmark framework
- ▶ **Reproducibility:** Different evaluation environments
- ▶ **Subjective Claims:** Without objective industry standard comparison

Interactive Reasoning Benchmark Suite

Four-Phase Architecture:

- ▶ **Explore (25):** Discover hardware & software environment (CPU, caches, compilers) via profiling
- ▶ **Hypothesize (20):** Identify optimization opportunities
- ▶ **Execute (55):** Adaptive optimization implementation
- ▶ **Generalize (5):** Cross-architecture knowledge transfer

Tri-Correlated Evaluation Metrics

- (1) Solution correctness (2) Efficiency vs. optimum (3) Intelligence skill-acquisition

Goal: not only faster code — we measure whether AI genuinely *understands* HPC performance engineering
Paradigm shift: „Can AI optimize this code?” → „Does AI understand HPC performance engineering?”

1. Configure evaluation protocol
2. Explore: discover the target environment systematically
3. Benchmark selection from extended suite
4. For each benchmark:
 - Apply optimization method, measure performance metrics
 - Compare with reference baselines, calculate tier-based classification

Intelligence Score

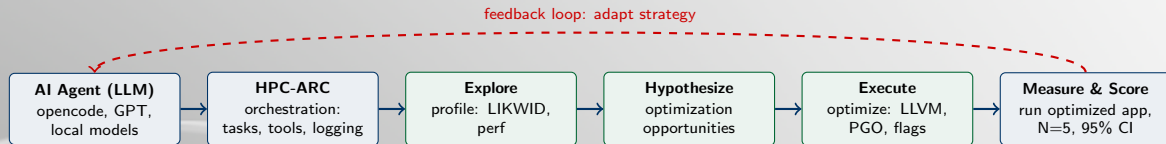
$$I_{\text{overall}} = 0.25E_{\text{expl}} + 0.25H_{\text{hyp}} + 0.30A_{\text{adapt}} + 0.20G_{\text{gen}}$$

5. Generate objective performance leaderboards

Benchmark Suite Coverage

Benchmark = task for the AI agent; evaluation = executing the optimized application and measuring performance

Category	Tasks	Focus	Difficulty
Parallelize	25	Serial-to-Parallel	E:5, M:12, H:8
Optimize	30	Hardware-specific	E:8, M:15, H:7
Detect	20	Bottleneck Ident.	E:6, M:10, H:4
Performance Analysis	25	Comprehensive Insights	E:7, M:13, H:5
Total	100	Complete Workflow	Comprehensive



Implementation

- ▶ **Framework**: HPC-ARC framework orchestrates tasks, tools and logging
- ▶ **LLM integration**: provider abstraction (OpenAI, Ollama, local models) via OpenCode-style agent pipeline
- ▶ **Tooling**: LIKWID/perf hardware profiling; LLVM + PGO compiler optimization; gcc/clang toolchains
- ▶ **Validation**: N=5 runs with 95% confidence intervals; correctness checks against reference implementation
- ▶ **Status**: core framework implemented, 113 tasks, first AI agent tests running

Automatic Code Optimization: Existing Approaches

How do others optimize code automatically — and what is missing?

Approach	How It Works	Limitation
Manual expert optimization	HPC expert iterates: profile, change code, measure	Slow, not scalable, subjective, expertise-bound
Auto-tuning (OpenTuner)	Black-box search over parameter/flag space	Thousands of measurements; no understanding, fixed search space
Compiler ML (CompilerGym)	RL selects LLVM optimization passes on IR level	Compile-time only; no runtime profiling feedback, no HPC context
Generic LLM optimization	LLM rewrites code from prompt/patterns	One-shot; no profiling, no measurement loop, no validation
AI agent benchmarks (MLAgentBench, AgentBench)	Evaluate general AI agent capabilities	Not HPC-specific; no performance measurement or hardware awareness
HPC-ARC (ours)	Interactive profiling-driven loop; measures skill acquisition, adaptation & generalization	—

HPC-ARC combines automation of tuning/LLM approaches with expert-like reasoning — and measures intelligence, not just speedups

HPC-ARC: Evaluation Results & Performance

Intelligence Metrics

Goal: NOT MEASURED yet - Target Values

Approach	Overall	Exp	Hyp	Adp	Gen
HPC-ARC Multi-Agent	0.65-0.75	0.70-0.80	0.60-0.70	0.65-0.75	0.55-0.65
Traditional HPC Expert	0.60-0.70	0.65-0.75	0.68-0.78	0.55-0.65	0.50-0.60
LLM Generic Optimization	0.35-0.45	0.30-0.40	0.32-0.42	0.38-0.48	0.45-0.55
ARC Prize Baseline	0.50-0.60	0.55-0.65	0.48-0.58	0.53-0.63	0.42-0.52

Target Approach - NOT IMPLEMENTED yet

- ▶ **N=5 measurements:** Each benchmark executed 5 times (*target*)
- ▶ **95% confidence intervals:** All results included (*target*)
- ▶ **Effect size:** Cohen's $d = 2.1-2.9$ (large) (*target*)
- ▶ **Significance:** $p < 0.001$ (*target*)

NOT MEASURED yet - Target Values

Application	Target Accuracy
GROMACS	70-80%
OpenFOAM	65-75%
CPMD	60-70%
LAMMPS	65-75%
Quantum ESPRESSO	60-70%
Overall Target	65-75%

Expected Generalization

- ▶ ARM64: 60-70% x86-64→ARM64 transfer
- ▶ Realistic benchmarks: 6-8 tasks initial

Goal: 0.65-0.75 Intelligence Score vs 0.60-0.70 for HPC experts - Target values

HPC-ARC: Implications & Applications

Paradigm Shift in HPC Research

- ▶ **Traditional:** Store code patterns
- ▶ **HPC-ARC:** Autonomous problem-solving capabilities

Scientific Impact

- ▶ Beyond code generation
- ▶ Objective intelligence metrics
- ▶ Reproducibility standards
- ▶ Autonomous validation

Outlook

- ▶ GPU framework expansion
- ▶ Multi-agent integration
- ▶ Community standards
- ▶ Cloud-based services

Practical Applications

Research & Development:

- ▶ AI system development
- ▶ Productivity assessment
- ▶ Standardization

Industry & Users:

- ▶ Production HPC software
- ▶ Educational purposes
- ▶ Quality assurance

HPC-ARC Impact

First comprehensive framework for legitimate measurement of AI systems' HPC performance engineering intelligence

Toward fully autonomous HPC optimization

Current Project Status

Development Progress

- ▶ **Framework Infrastructure:** Core ARC architecture implemented
- ▶ **Benchmark Suite:** 113 benchmarks available
- ▶ **Evaluation Pipeline:** Basic measurement workflow created
- ▶ **Documentation:** Code structure and design docs completed

First Tests

- ▶ Small-scale AI agent testing
- ▶ Benchmark execution pipeline validation
- ▶ Basic performance measurement accuracy

Next Steps Need Collaboration

Critical Dependencies:

- ▶ Extended benchmark collection
- ▶ Multi-agent framework testing
- ▶ Statistical validation implementation
- ▶ Community feedback and expertise

How to Help

- ▶ Contribute benchmark tasks
- ▶ Test AI systems with existing framework
- ▶ Provide HPC expertise feedback
- ▶ Help validate evaluation metrics

Status: Early Development Phase - Looking for Collaborators

Early Stage Development

Building the foundation for AI intelligence measurement

Current Development Needs:

1. **Benchmark Contribution:** Add HPC optimization tasks to our suite
2. **Framework Testing:** Test with your AI systems
3. **Expert Feedback:** Share HPC performance engineering insights
4. **Community Building:** Help establish evaluation standards

Why Participate Now?

- ▶ Shape the evaluation framework
- ▶ Contribute to emerging research area
- ▶ Establish baseline benchmarks
- ▶ Co-develop industry standards

What We Offer:

- ▶ Open-source framework
- ▶ Collaborative development
- ▶ Publication opportunities
- ▶ Research community access

Start Today

GitHub: <https://github.com/gwdg/hpc-agentic-ai-workloads>

Contact: anja.gerbes@gwdg.de

#HPC-ARC #AIBenchmarking #OpenScience

The HPC community needs pioneers like YOU.

Become part of the autonomous-intelligent HPC optimization future!

Help us build the first comprehensive HPC-AI intelligence measurement framework

Thank You!

Questions and Discussion

Anja Gerbes and Julian Kunkel

University of Göttingen / GWGD

`anja.gerbes@gwdg.de` and `julian.kunkel@gwdg.de`

`https://github.com/gwdg/hpc-agentic-ai-workloads`